

大專生計畫 v4 — 沉浸式籃球觀賽系統：從轉播戰術解析到智慧眼鏡 AR

112703028 資訊三 吳亭翰

指導教授：廖文宏

一、摘要

本計畫旨在開發一套**沉浸式籃球觀賽與戰術解析系統**，最終目標是透過智慧眼鏡（如 Ray-Ban Meta）結合邊緣運算，讓現場觀眾在不打斷觀賽流的前提下，即時獲得專業球評等級的戰術解說（如 Pick-and-Roll、雙重掩護等）。

為達此目標，本計畫採取**兩階段**研究路線：

- **第一階段（已完成 / 進行中） — 戰術偵測核心演算法驗證**

以伺服器端 GPU 訓練並驗證 PnR 偵測核心 pipeline：**SAM2 跨幀追蹤 + 自訓 multi-class YOLO (handler_detector.pt) + sports.TeamClassifier (SigLIP + UMAP + K-means) + 自行設計的 ScreenNetPairwise (LSTM + Transformer + Pairwise Head)**。在 164 段人工標注 clip 上達成 ball_handler 偵測準確率 0.94、screener F1 0.89。

- **第二階段（規劃中） — 邊緣部署 + 智慧眼鏡 AR 整合**

將驗證過的模型透過 **INT8 量化感知訓練 (QAT)** 部署至 NVIDIA Jetson Nano，搭配 Ray-Ban Meta 第一人稱影像，以 D-ROI + 非同步 pipeline 達到毫秒級 AR 同步，最終輸出類 NBA 2K 風格的 HUD 與戰術重現。

- **第三階段（規劃中） — LLM 戰術語義解讀**

將前兩階段產出的結構化資料（球員軌跡、掩護事件、投籃事件、傳球路徑）作為 prompt context 輸入大型語言模型（如 GPT-4o、Claude、Gemini），由 LLM 生成**自然語言戰術分析**：辨識戰術名稱（Spain P&R、Horns、Stagger 等）、解釋執行意圖、評估成敗原因。這層讓系統從「偵測戰術」升級到「**理解並講解戰術**」，等同 AI 球評。

關鍵字：basketball tactical analysis、Pick-and-Roll detection、SAM2、SigLIP、ScreenNet、Homography、Edge AI、AR、Smart Glasses、Jetson Nano、LLM、AI commentator

二、研究動機與研究問題

1. 研究動機

對「一日球迷」而言，現代籃球已進入精確的空間科學時代。**西班牙擋拆 (Spain Pick & Roll)**、**雙重掩護 (Double Drag)**、**Horns Set** 等戰術涉及多名球員在 2-3 秒內的空間與時序切換，對一般觀眾如同混亂的跑位。

目前的痛點：

1. **視覺資訊與現場體驗的斷層** — 看現場想知道命中率要低頭看手機 App，打斷觀賽
2. **戰術理解的認知門檻** — 沒有專業球評難以理解擋拆走位
3. **第一人稱分析的技術瓶頸** — 智慧眼鏡視角晃動 + 算力受限，邊緣端要做精準分析非常難

本計畫想透過 **AI 自動戰術偵測 + AR 視覺化** 降低這些門檻。但「戰術偵測」本身就是個獨立難題 — 即使在伺服器端用乾淨的轉播影片，多球員追蹤、角色辨識、掩護判定都還沒有成熟方案。因此本計畫先聚焦 **演算法核心驗證**，再推進到邊緣 / 眼鏡部署。

2. 研究問題

第一階段（演算法核心）

1. **跨幀穩定追蹤**：在頻繁遮擋（擋拆、卡位）下如何維持球員 ID 一致？
2. **角色與球權精準辨識**：能否不依靠球追蹤就準確判定「誰持球」、「誰是掩護者」？
3. **無監督球隊分類**：如何在不同配色、不同比賽中自動分辨進攻 / 防守隊？
4. **掩護對應關係預測**：如何從 10 幀軌跡判斷「j 為 i 做掩護」這種對應關係？

第二階段（邊緣 + AR 部署）

5. **算力瓶頸下的推理優化**：如何在 Jetson Nano 邊緣設備上，透過 QAT 與非同步 pipeline，達成觀賽流暢度的即時推理 (≥ 30 FPS) ？
6. **動態視角下的空間校準**：第一人稱視角有魚眼效果與劇烈晃動，如何用 Homography + ORB 補償達成穩定的球場標記點映射？
7. **AR 認知負荷設計**：HUD 資訊（命中率、體力條、戰術標記）如何分層，避免視覺過載？

第三階段（LLM 戰術解讀）

8. **結構化 → 語義轉換**：如何把球員軌跡 (10 player $\times$ T frame $\times$ 2 coords)、screener-handler 配對、shot event 等多模態結構資料，轉成 LLM 能讀懂的 token 序列？
9. **戰術命名與解釋**：如何 prompt LLM 從一段擋拆事件 → 正確識別為 Spain P&R / Double Drag / Stagger，並產出觀眾能懂的中文解說？

10. **幻覺控制**：LLM 對動作描述容易腦補（如「球員飛快奔跑」），如何用 retrieval-augmented generation（RAG）+ 結構化事實校驗確保解說準確？

三、文獻回顧與探討

球員追蹤

技術	代表文獻	優點	缺點	本計畫採用？
ByteTrack	Zhang et al. (2022)	速度快、輕量	球員交錯時 ID 跳動	第一版用過後棄用
SAM2 Video Predictor	Ravi et al. (2024)	mask-based 追蹤、ID 穩定	計算量較大	 第一階段採用
SpatialTracker	Wang et al. (2023)	像素級精度	大面積遮擋時失效	不採用

戰術 / 動作辨識

技術	代表文獻	優點	缺點
ST-GCN	Yan et al. (2018)	骨架結構明確	對長時序複雜戰術辨識力弱
Video Swin Transformer	Liu et al. (2022)	層級化注意力	對標注資料量需求高
ACA-Net	Zhang et al. (2024)	自適應環境感知	需針對特定球場預訓練
ScreenNetPairwise (本計畫)	自行設計，基於 NETS 論文延伸	直接針對 5v5 球員配對學習掩護關係，輕量	需自行標注 PnR 資料

ScreenNetPairwise 架構：LSTM 編碼球員軌跡 → role embedding（handler / offense / defense）→ Transformer Encoder → Pairwise Head 輸出 20 對的 screen 機率。

球員角色與球隊識別

方法	優缺點
球追蹤 + 最近球員 = 持球者	球被遮擋時 handler 跳人，不穩定
多 class YOLO 直接偵測 player-in-possession (本計畫)	視覺判斷誰拿球，準確率 0.94， 不靠球追蹤
fashion-CLIP zero-shot	需指定 prompt，對相近配色不穩
SigLIP + UMAP + K-means (本計畫)	無監督、視覺特徵 cluster

邊緣運算與量化（第二階段）

- Jacob et al. (2018) — 量化感知訓練（QAT）將模型轉 INT8，運算速度大幅提升
- Mittal (2020) — Jetson Platform 深度學習部署策略
- Grauman et al. (2022) — Ego4D 第一人稱視覺資料集，提供第一人稱訓練樣本

AR / 智慧眼鏡

- Hervas et al. (2019) — 運動 AR 設計準則，強調「情境感知」避免資訊過載
- Zaccardi et al. (2023) — HoloLens2 即時 AI 推論案例（本計畫不採 HoloLens 因封閉系統與算力限制）

多 screener 戰術發現

實際分析 COCO 訓練資料發現 **43/164（26%）** 的 clip 同時有 **2-3 個 screener**（horns / staggered / double drag）。本計畫修改訓練資料生成邏輯，將「只取最多票的 1 個 screener」改成依「最擁擠那幀的 screener 數」決定保留幾個，pair label 從 Active pairs=1/20 提升到 3/20。

四、研究方法及步驟

第一階段：戰術偵測核心 pipeline

轉播影片

↓

[1] YOLO 第一幀偵測 + SAM2 跨幀傳播 → 穩定 player_id

↓

[2] handler_detector.pt (multi-class YOLO) → 每幀偵測 ball_handler bbox
→ IOU 配對 SAM2 player_id → 每幀 handler 球員

↓

[3] CourtKeypointDetector → Homography → 球員場上座標 (feet)

↓

[4] sports.TeamClassifier (SigLIP + UMAP + K-means) → 無監督分隊
+ YOLO defender class 補強 (解決 6vs4 team imbalance)

↓

[5] ScreenNetPairwise (10 幀 sliding window) → 對每個進攻候選人預測 screen pair

↓

[6] 視覺化輸出：HANDLER 綠框 / SCREENER 紅框 / 信心分數 / 事件段

1.1 SAM2 穩定追蹤

- `_pick_init_frame()` 找前 30 幀中球員偵測最多的當 init，避免黑屏 / scene 切換污染
- 反向 propagate — 若 init frame 不是第 0 幀，補跑 `reverse=True`

1.2 球員角色偵測（自訓 multi-class YOLO）

- 基於既有 COCO 標注（ball_handler、screener、defender、others）訓練 4-class YOLOv8
- 訓練結果：整體 mAP50 = 0.885；**ball_handler 準確率 0.94**

1.3 無監督分隊（sports.TeamClassifier）

- 每 30 幀抓所有 player bbox，縮 40% 取中央球衣區域
- SigLIP encoder → UMAP → K-means k=2，一次性 fit 整段影片
- YOLO + SigLIP 融合：YOLO 整段認為是 defender 的 player_id 強制改判 defense

1.4 ScreenNetPairwise

- Slot 順序：[handler, screener, other_offx3, defensex5]
- 對每個進攻候選人 c：放到 slot 1，其他進攻補 slot 2~4，跑模型取 pair (0,1) 分數
- 最高分若 $\geq$ threshold → 預測為 screener

3 階段訓練：

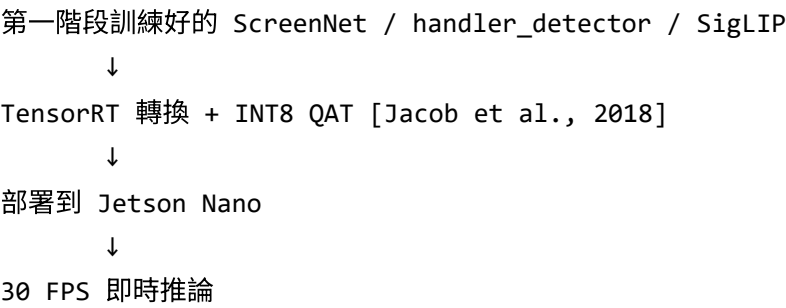
階段	資料	Loss
Step 1: Pretrain	NBA 公開 tracking 資料，軌跡預測自監督	MSE
Step 2: Weak finetune	規則自動標的 weak labels	BCE
Step 3: Manual finetune	164 clip 人工標注 + SAM2 + COCO 投票	Pairwise BCE

訓練結果：

模型	AUC	F1 @ 0.5	Best F1	Active pairs
v1 (single screener)	0.9217	0.8889	0.9031 @ 0.70	1/20
v2 (multi screener)	0.9633	0.5524	0.5852 @ 0.73	3/20

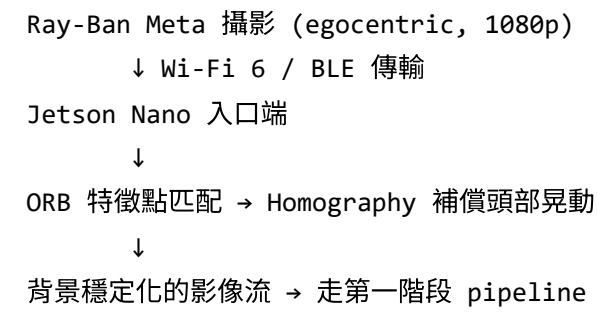
第二階段：邊緣部署 + 智慧眼鏡 AR

2.1 模型輕量化與邊緣部署



- **QAT 對象**：handler_detector.pt (4-class YOLO，主要計算瓶頸)
- **保留 FP16**：SigLIP encoder (精度敏感)
- **算子融合**：合併連續的 Conv + BN + ReLU

2.2 第一人稱視角影像穩定化



- 訓練資料增強：用 Ego4D dataset [Grauman et al., 2022] 模擬第一人稱視角晃動
- 平面映射仿射變換對 NCAA / Roboflow 廣播影片做「Ego-Simulation」，產生第一人稱風格訓練樣本

2.3 非同步 AR Pipeline

Thread 1: 影像擷取 + SAM2 tracking (30 FPS)

Thread 2: handler_detector + ScreenNet 推論 (10 FPS)

Thread 3: HUD 渲染 (60 FPS)，用最新 tracking + 緩存 inference

- HUD 不等推論，用最新 tracking 確保視覺同步
- 推論結果延遲 100-200ms 才更新，避開因等待造成的視覺殘影

2.4 AR HUD 設計

依 Hervas et al. (2019) 的情境感知原則：

HUD 元素	觸發條件	視覺呈現
球員姓名 + 命中率	觀眾凝視該球員 > 1 sec	浮動 tag 跟著球員腳底
體力條	持球者持續顯示	球員腳下橢圓圈
Screen 標記	ScreenNet 偵測到掩護	紅色圓柱 + 箭頭，2 秒淡出
戰術名稱	Spain P&R / Double Drag 識別	螢幕邊緣中文標籤

2.5 賽後戰術回顧模式

賽後階段算力不受限，可以離線跑：

- 完整 ScreenNet v2 (multi-screener)

- 用第一階段所有 component 重新跑一遍最佳化結果
- 輸出戰術重現影片（圖二風格的箭頭 + 標記）

第三階段：LLM 戰術語義解讀

前兩階段的輸出都是**結構化數值**（player_id、bbox、座標、screen score、shot result）。觀眾要看的不是表格，而是**「教練在講什麼戰術」**。第三階段把這些結構資料餵給 LLM，由它組合成自然語言解說。

3.1 結構化事件 → LLM-friendly token

每段比賽片段先彙整成一個 JSON event timeline：

```
{
  "clip_id": "screen_13",
  "duration_sec": 5.0,
  "players": [
    {"id": 5, "team": "offense", "role": "handler", "trajectory": [...]},
    {"id": 9, "team": "offense", "role": "screener", "trajectory": [...]},
    {"id": 3, "team": "defense", "role": "defender_of_screen", "trajectory": [...]},
    ...
  ],
  "events": [
    {"t": 1.2, "type": "screen_set", "screener": 9, "handler": 5, "court_pos": [85, 25]},
    {"t": 2.4, "type": "ball_pass", "from": 5, "to": 9},
    {"t": 3.1, "type": "shot", "shooter": 9, "result": "made", "type_": "layup"}
  ],
  "court_pos_summary": {
    "screen_location": "top_of_key",
    "shooting_zone": "paint"
  }
}
```

軌跡可以**降採樣**並轉成自然語言摘要（如「handler 從右側 45 度切入禁區」）避免 token 爆炸。

3.2 LLM Prompt 設計

你是專業籃球戰術分析師，根據以下事件 timeline，
辨識這段是哪種戰術（從：Spain P&R / Double Drag / Horns / Stagger / 其他），
並用 30 字內中文解釋執行意圖。

[Event timeline JSON]

採用 **Few-shot prompting + RAG**：

- Few-shot：給 LLM 看 10-20 個標注好的範例
- RAG：當 LLM 不確定戰術名稱時，去檢索「戰術知識庫」（戰術定義 + 典型動作描述）

3.3 LLM 選擇

模型	場景	部署
Claude / GPT-4o	賽後離線分析、高品質解說	Cloud API
Gemini Nano / Phi-3	即時 AR 短解說 (< 50 字)	Edge / Mobile
Llama 3 8B (本地)	開發階段不耗 API 額度	本地 GPU

第一階段先用 Claude 確保品質，再壓縮到 small model 做 edge 部署。

3.4 幻覺校驗

LLM 容易腦補出沒發生的事（「精彩的後撤步三分」其實只是普通跳投）。校驗策略：

- **Grounding check**：LLM 輸出後做 keyword extraction，每個動作名稱對應回原始 event。沒對應到原始資料的 → 標記為「不確定」或刪掉
- **Structured output**：強制 LLM 用 JSON 回（如 `{"tactic_name": ..., "confidence": ..., "evidence_events": [...]}`），confidence 低就回 "未知戰術"
- **人工標 ground truth**：在第三階段保留一組標好的 PnR clip → 計算 LLM tactic naming 的 accuracy

3.5 輸出形式

場景	輸出
賽後分析報告	整場比賽各 PnR 的中文解說 + 戰術圖（圖二風格）

場景	輸出
即時 AR HUD	偵測到 PnR 後，眼鏡邊緣浮現「Spain P&R」+ 1 行解說
教練 dashboard	各戰術成敗率 / 偏好區域 / 球員配合熱度圖

五、預期結果

第一階段

-  端到端 pipeline 完成 → 從轉播影片輸出帶 PnR 標記的視覺化影片
-  ball_handler 偵測準確率 > 90% (已達 94%)
-  掩護偵測 F1 > 0.85 (v1 已達 0.89)
-  支援多 screener 場景 (v2 已實作)
-  完整 confusion matrix 與誤判 case study

第二階段

-  Jetson Nano 上 INT8 量化後達 30 FPS 推論
-  Ray-Ban Meta + Jetson Nano 整合 demo (戶外籃球場實測)
-  AR HUD 視覺同步延遲 < 100ms
-  戶外實測 PnR 偵測準確率不低於第一階段 (測試集 F1 > 0.7)

第三階段

-  LLM 戰術命名準確率 > 75% (在 5 種主流戰術上)
-  自然語言解說可讀性人類評分 > 4/5 (盲測)
-  幻覺率 < 10% (grounding check 通過率)
-  Edge LLM (如 Phi-3) 做 30 字內即時短解說，<200ms 延遲

六、目前進度

已完成 (第一階段)

-  SAM2 + multi-class YOLO + sports.TeamClassifier + ScreenNetPairwise 完整 pipeline

- ✓ 164 clip 人工標注 + npz 生成
- ✓ 3 階段訓練 (pretrain → weak finetune → manual finetune)
- ✓ 多 screener 支援 (v2 訓練)
- ✓ 影片實測 + 視覺化

進行中（第一階段優化）

- ☐ SAM2 mask drift 修正 (YOLO 驗證每幀 bbox)
- ☐ YOLO + SigLIP 加權融合 (解決 team 邊緣 case)
- ☐ defender vs screener 軌跡混淆問題 (YOLO override)

規劃中（第二階段）

- ☐ handler_detector.pt INT8 量化 (TensorRT)
- ☐ Jetson Nano 部署測試
- ☐ Ray-Ban Meta 影像串流接入
- ☐ AR HUD 渲染 pipeline
- ☐ 戶外實測

規劃中（第三階段）

- ☐ 結構化 event timeline 格式設計
- ☐ Claude / GPT-4o few-shot prompt 設計
- ☐ 戰術知識庫建構 (5 種主流戰術定義 + 動作描述)
- ☐ Grounding check 自動驗證 pipeline
- ☐ Edge LLM (Phi-3 / Gemini Nano) 壓縮部署

七、需要指導教授指導內容

第一階段

1. **多 screener pair label 平衡**：v2 active pairs=3/20 但 F1 反而下降，是否需要調整 loss weighting 或 negative sampling？
2. **多模態 fusion 策略**：SigLIP team prob、YOLO class logit、ScreenNet pair score 三者最佳融合方式？
3. **SAM2 mask drift 解法**：是否需要每 N 幀重新 init，或加入 YOLO 驗證 + Hungarian 配對？

第二階段

4. **Jetson Nano 上的精度 / 速度平衡**：哪些 model 適合 INT8，哪些必須保留 FP16？
5. **Ray-Ban Meta SDK 限制**：智慧眼鏡是否能即時傳輸高解析影像到 Jetson Nano？bottleneck 在哪？
6. **第一人稱訓練資料**：如何從 broadcast 影片有效模擬 egocentric 視角？

第三階段

7. **LLM prompt 設計**：要把多少結構資料 / few-shot example 餵 LLM，才能在 token limit 內達到最高戰術辨識率？
8. **幻覺評估與校驗**：用什麼自動 metric 量化 LLM 解說的可信度？是否需要訓練專門的 grounding classifier？
9. **領域知識嵌入**：戰術定義要存成 vector DB（RAG）還是直接寫進 system prompt？哪種對 small LLM 更有效？

八、檔案結構

```
basketball_analysis/
├─ nba_data_pipeline/
│   ├── convert_manual.py          # COCO → 訓練 npz (v2 多 screener)
│   ├── finetune.py                # ScreenNet 訓練
│   ├── inference.py               # 完整 inference pipeline
│   ├── model.py                   # ScreenNetPairwise 定義
│   ├── coco_to_yolo.py            # COCO → YOLO 格式轉換
│   ├── test_ball_acquisition.py   # handler 偵測驗證
│   ├── PROGRESS.md
│   ├── checkpoints/               # pretrain / finetune_weak / finetune_manual
│   └─ output_manual/*.npz
├─ team_assigner/team_assigner.py  # sports.TeamClassifier
├─ utils/video_utils.py            # cv2 + imageio fallback
├─ models/
│   ├── player_detector.pt         # YOLO 單 class
│   ├── handler_detector.pt        # multi-class YOLO ★
│   ├── court_keypoint_detector.pt
│   └─ sam2.1_hiera_large.pt
└─ yolo_screen_dataset/            # YOLO 訓練資料
```

九、參考文獻

追蹤與分割

1. Ravi, N., et al. (2024). SAM 2: Segment Anything in Images and Videos. arXiv:2408.00714.
2. Zhang, Y., et al. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *ECCV*.

戰術 / 動作辨識

3. Yan, S., Xiong, L., & Lin, D. (2018). Spatial temporal graph convolutional networks for skeleton-based action recognition. In *AAAI*.
4. Liu, Z., et al. (2022). Video Swin Transformer. In *CVPR*.
5. Zhang, Y., Zhang, F., Zhou, Y., & Xu, X. (2024). ACA-Net: Adaptive context-aware network for basketball action recognition. *Frontiers in Neurorobotics*, 18, 1471327.
6. Wu, Y., et al. (2020). Fusing motion patterns and key visual information for semantic event recognition in basketball videos. *Information Processing & Management*, 57(3).
7. Cunha, P., et al. (2019). Towards automatic basketball tactical analysis. In *ICIP*.

球員 / 球隊識別

8. Zhai, X., et al. (2023). Sigmoid Loss for Language Image Pre-Training (SigLIP). In *ICCV*.
9. McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection. arXiv:1802.03426.
10. Roboflow. (2024). sports package. <https://github.com/roboflow/sports>
11. Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>

邊緣 AI / 智慧眼鏡（第二階段）

12. Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *CVPR* (pp. 2704-2713).
13. Mittal, S. (2020). A Survey on Optimized Implementation of Deep Learning Models on the NVIDIA Jetson Platform. *Journal of Systems Architecture*.
14. Zaccardi, S., Frantz, T., Beckwée, D., & Jansen, B. (2023). On-device execution of deep learning models on HoloLens2 for real-time augmented reality medical applications. *Sensors*, 23(21), 8698.

15. Grauman, K., et al. (2022). Ego4D: Around the world in 3,000 hours of egocentric video. In *CVPR* (pp. 18995-19012).
16. Hervas, R., et al. (2019). Augmented reality in sports: A survey of the state of the art. *Applied Sciences*, 9(22).
17. Syberfeldt, A., et al. (2017). Augmented Reality at the Industrial Shop-floor.

LLM 戰術解讀（第三階段）

18. OpenAI. (2024). GPT-4o System Card. <https://openai.com/gpt-4o>
19. Anthropic. (2024). Claude 3.5 Sonnet Model Card.
20. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
21. Abdin, M., et al. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219.
22. Brown, T., et al. (2020). Language Models are Few-Shot Learners. In *NeurIPS*.

經典基礎

23. Vaswani, A., et al. (2017). Attention is all you need. In *NeurIPS*.
24. Zheng, S., et al. (2016). Egocentric basketball motion planning from a single first-person image. In *CVPR*.